

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA1711

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I Kishore S.P (192511081) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

(a) Load, Pre-process and Split Dataset

Using the Iris dataset:

Python

Run

```
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier, export_text
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
import pandas as pd
```

```
iris = load_iris()
```

```
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df["target"] = iris.target
```

```
print(df.head())
print(df.dtypes)
```

```
X = df.drop("target", axis=1)
y = df["target"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.30, random_state=42, stratify=y
)
```

Preprocessing: Iris has no missing values and no categorical features, so no encoding is required.

(b) Build Decision Tree

Python

Run

```
model = DecisionTreeClassifier(
    criterion="gini",
    random_state=42
)
```

```
model.fit(X_train, y_train)
```

```
print(export_text(model, feature_names=list(X.columns)))
```

The root node is generally petal length (cm) or petal width (cm), depending on the exact split/training configuration.

The root is selected because it provides the largest reduction in Gini impurity, giving the best separation of Iris classes.

(c) Evaluation

Python

Run

```
y_pred = model.predict(X_test)
```

```
print("Accuracy:", accuracy_score(y_test, y_pred))
```

```
print("Confusion Matrix:")
```

```
print(confusion_matrix(y_test, y_pred))
```

```
print("Classification Report:")
```

```
print(classification_report(
```

```
    y_test, y_pred,
```

```
    target_names=iris.target_names
```

```
))
```

Typical Iris Decision Tree performance is around 90–100% accuracy.

Comment: The model performs very well because Iris classes, especially *setosa*, are relatively easy to separate using petal measurements.

Misclassified instances

The exact misclassified rows depend on the train/test split. They can be identified using:

Python

Run

```
misclassified = X_test[y_test != y_pred]
```

```
print(misclassified.head(2))
```

These are usually difficult cases where Iris versicolor and Iris virginica have overlapping measurements.

Conclusion: Decision Tree is simple, interpretable, and highly effective for the Iris classification problem.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

(a) Grid World Environment

Use a **4×4 grid** with states –.

- **Actions:** Up (U), Down (D), Left (L), Right (R)
- **Goal:** State 15 $\rightarrow$ +10
- **Normal step:** -1
- **Obstacle:** -5
- **Initial Q-table:** all zeros

Hyperparameters:

where:

- = learning rate
- = discount factor
- = exploration probability

Q-learning update:

(b) Training for 100 Episodes

Python

Run

```
import numpy as np
```

```
Q = np.zeros((16, 4))
```

```
alpha, gamma, epsilon = 0.1, 0.9, 0.1
```

```
# Actions: 0=Up, 1=Down, 2=Left, 3=Right
```

```
for episode in range(100):
```

```
    state = 0
```

```
    while state != 15:
```

```
        if np.random.rand() < epsilon:
```

```
            action = np.random.randint(4)
```

```
        else:
```

```
            action = np.argmax(Q[state])
```

```
        # Environment transition implemented here
```

```
        next_state = state # replace with grid transition
```

```
        reward = -1      # +10 goal, -5 obstacle
```

```
Q[state, action] += alpha * (
    reward + gamma * np.max(Q[next_state]) - Q[state, action]
)
```

```
state = next_state
```

Example observation of Q-table evolution:

Episode	State	U	D	L	R
1	0	0	0	0	0
1	5	0	0	0	0
50	0	-0.8	-0.4	-0.7	4.2
50	5	-0.6	5.1	-0.5	2.4
100	0	-0.9	-0.3	-0.8	6.8
100	5	-0.7	7.2	-0.6	3.1

Values are illustrative; actual values depend on the grid and random seed.

The Q-values gradually increase for actions leading toward the goal.

(c) Final Policy and Convergence

The optimal action is:

Example final policy:

→ → → ↓

↑ → → ↓

↑ → → ↓

↑ → → G

where **G = Goal**.

Cumulative reward can be plotted using:

Python

Run

```
import matplotlib.pyplot as plt
```

```
plt.plot(cumulative_rewards)
```

```
plt.xlabel("Episode")
```

```
plt.ylabel("Cumulative Reward")
```

```
plt.title("Q-Learning Convergence")
```

```
plt.show()
```

Convergence

Initially, rewards are low because the agent explores randomly. After repeated episodes, the agent learns the shortest safe path to the goal, so the cumulative reward generally **increases and becomes more stable**.

Conclusion: Q-Learning successfully learns an optimal/near-optimal policy through repeated interaction without being given the correct path in advance.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation